Section 09: デプロイ & 発展

作ったアプリを世界へ公開する

ローカル環境（自分のPC）で動かしていたアプリケーションをインターネット上に公開し、誰でも使えるようにする「デプロイ」の概念と、今後の発展学習について学びます。

1. 「デプロイ」とは何か？

これまで皆さんが書いてきたコードは、皆さんのPC（`localhost`）の中でしか動いていませんでした。

- **デプロイ (Deploy)**: 開発したアプリケーションを、実際のサーバー（インターネット上のコンピュータ）に配置し、外の世界からアクセスできる状態にすること。

フロントエンド (React) とバックエンド (Hono) は、**別々のサービス** にデプロイします。

	デプロイ先	役割
フロントエンド	Firebase Hosting	HTML / CSS / JS を配信する
バックエンド	Cloudflare Workers	API（データの読み書き）を動かす

2. Cloudflare を使ったデプロイ

現代のWeb開発では、複雑なサーバー設定をプログラマー自ら行うことは減っています。今回は **Cloudflare** というプラットフォームを使って、驚くほど簡単にアプリを公開する仕組み（概念）を学びます。

なぜ Cloudflare を使うのか？

- **無料枠が強力**: 個人開発や小〜中規模なアプリなら、ほとんど無料枠で収まります。
- **エッジネットワーク**: 世界中にあるサーバー（エッジ）のうち、一番ユーザーに近い場所からデータが返されるため、通信が非常に高速です。

3. バックエンドのデプロイ: `wrangler`

バックエンド（Hono で作った API）は、**Cloudflare Workers** というサービスの上にデプロイします。
ここで使う魔法のコマンドが `wrangler` （ラングラー）です。

```
# デプロイするためのたった一つのコマンド
npx wrangler deploy
```

たったこれだけで、手元のコードがCloudflareのサーバーにアップロードされ、 `https://my-api.xxxx.workers.dev` のようなURLが発行されます。

`wrangler.jsonc` (または `wrangler.toml`) の役割

`wrangler deploy` コマンドを叩いた時、システムはどのように環境を構築すればいいかを知るために「設計図」を読みに行きます。

- アプリの名前は何か？
- どのデータベース (D1) を使うか？
- どのストレージ (KV や R2) に繋ぐか？

これらが書かれた設定ファイルが `wrangler.jsonc` です。このファイルに適切にIDを書き込むだけで、インフラの設定が完了します。(Infrastructure as Code: インフラのコード化)

4. フロントエンドのデプロイ: Firebase Hosting

フロントエンド（React / Vite で作った画面）は、Google の **Firebase Hosting** にデプロイします。

バックエンドと違い、フロントエンドは「プログラムを動かす」のではなく、「完成した HTML, CSS, JS のファイル群を配る」だけなので仕組みが少し違います。

Step 1: Firebase CLI をインストールする

```
npm install -g firebase-tools
```

Firebase のコマンドラインツールを PC にインストールします。これが `wrangler` のフロントエンド版だと思ってください。

Step 2: ログインして初期設定をする

```
# Google アカウントでログイン
firebase login

# プロジェクトの初期設定（対話形式で進む）
firebase init hosting
```

`firebase init hosting` を実行すると、いくつか質問されます。

質問	答え方
「What do you want to use as your public directory?」	`dist` と入力（Vite のビルド出力先）
「Configure as a single-page app?」	**Yes** を選択（React は SPA なので）
「Set up automatic builds with GitHub?」	ここでは **No**（後で設定できる）

Step 3: ビルド & デプロイ

```
# 1. 開発用のコードを本番用の最適化されたファイル群に変換する（ビルド）
npm run build

# 2. Firebase Hosting にアップロードする
firebase deploy --only hosting
```

完了すると `https://<プロジェクト名>.web.app` のような URL が発行され、フロントエンドが世界に公開されます。

5. デプロイの自動化と GitHub 連携

いま学んだ `firebase deploy` や `wrangler deploy` を使うと「自分のPCから」世界に公開できます。これでも十分すごいことですが、実はもっと簡単な方法があります。

それが **「GitHub との連携」** です。

1. 作ったコードを GitHub に上げる
2. Firebase コンソール（Web 管理画面）の **Hosting** 設定画面から、GitHub リポジトリを連携する
3. 以降、**コードを書いて GitHub に push するだけで、Firebase が自動的に最新版を公開してくれる**

> Cloudflare Workers（バックエンド）側も同様に、Cloudflare ダッシュボードから GitHub 連携を設定できます。

この仕組みを使えば、コマンドを手打ちする手間すらなくなります。初学者のうちはこの「GitHub 連携」を使うのが一番楽で安全な方法です！

6. 次のステップ：次に何を学ぶべきか？

Part 1〜8 を終えた皆さんが、これから実践で活躍するために広げていくべき分野をいくつか紹介します。

1. 認証とセキュリティ

「誰がログインしているか」を管理するシステムです。
キーワード

- **Cookie と Session**: ユーザーの記憶をどう保持するか
- **JWT (JSON Web Token)**: 現代的な認証の仕組み
- API にアクセス制限をかける方法（ミドルウェアの活用）

2. より複雑な状態管理

React の `useState` だけでなく、アプリ全体の巨大なデータを扱う方法。

- **カスタムフック**: 複雑な処理を部品化する
- **SWR / TanStack Query**: 非同期データ取得とキャッシュの専門ライブラリ

3. スタイリングの極意

素のCSSを卒業し、よりモダンで効率的なデザイン手法の習得。

- **Tailwind CSS**: クラス名を並べるだけで爆速でデザインができる今の主流
- UIコンポーネントライブラリ (**shadcn/ui** など) の活用

4. チーム開発の手法

一人ではなく、複数人でアプリを作るためのルールと道具。

- Git の高度な使い方 (ブランチ戦略, コンフリクトの解消)
- PR (Pull Request) を使ったコードレビューの相互実施

7. おわりに

「完璧を目指すより、まずは動かすこと」

プログラムは、作っている最中はエラーだらけで心が折れそうになることもあります。ですが、画面に文字が出た瞬間、データベースに値が入った瞬間の「動いた！」という達成感は何物にも代えがたいものです。

研修の内容は一気にすべて理解できなくても大丈夫です。「あ、これ研修でやった気がする」と思い出して、資料や過去のコード、公式ドキュメントに戻ってこられるようになることが、何よりの成長です。

全10回を通じて、皆さんは以下のスキルを習得しました！

1. **Git/GitHub**: チーム開発の土台
2. **TS/TSX**: 安全で強力なプログラミング言語
3. **React**: モダンで高速な UI 構築
4. **Hono/Cloudflare**: スケールするバックエンドとデータベース
5. **AI Integration**: 最新の知能を持ったアプリ開発

「まずは動くものを作る」。この楽しさを忘れずに、自分だけのオリジナルアプリの開発に挑戦してみてください！